Ans:- to the Question No: 1

a) <u>Brightness</u>:- Brightness refers to the overall Lightness or daarkness of an image. It is controlled by adding or substructing a constant value to all pixel intensity.

If brightness is increased → pixels shift toward white. If brightness is decreased → pixels shifted towards to black.

Mathematically:

$$g(x,y) = f(x,y) + \beta$$

Where,

$f(x,y)$ = original pixel intensity

β = brightness adjustment factor

positive → brighter

negative → darker

Contrast: Contrast refers to the difference between the darkest and brightest part of an image.

- High contrast → clear separation between dark and light area
- Low contrast → image looks dull/washed out (narrow intensity range)
- Controlled by multiplying pixel values by a scalling factor.

Mathematically:-

$$g(x,y) = \alpha \cdot f(x,y)$$

where,

$f(x,y)$ = original pixel intensity

α = Contrast factor.

$\alpha > 1$; increase contrast

$0 \leq \alpha \leq 1$; decrease contrast.

(b) Image restoration:- Image restoration is the process of recovering an image as original and clean from a degraded or noisy version.

* The goal is ~~is~~ to reverse the effects of blurring, noise, or distortions that occur during image capture, transmission, storage

* Unlike image inhancement, restoration tries to reconstruct the true original image using mathematical models.

Noise (in an image):- Noise is an image refers to unwanted random varience/variations of brightness or color values that degrade the visual quality.

It is usually introduced by sensor, transmission error, or environment factors.

Common type of image noise:-

(1) Gaussian noise (2) Salt and paper noise

(3) Speckle noise -

(c) Application of digital image processing:-

(1) Medical Imaging:

(i) MRI
(ii) CT Scans
(iii) Ultrasound image denoising
(iv) Cancer detection, organ segmentation

(2) Remote Sensing and satellite image
(3) Industrial and Manufacturing
(4) Forensics and Security
(5) Astronomy and Space Exploration
(6) Entertainment and Multimedia
(7) Traffic and transportation.

(d) Application of Colors Models

CMY (Cyan, Magenta, Yellow):-

(i) Mainly used in printing and publishing
(ii) printer use CMY (and CMYK with black) for producing hard copy.

YIQ (Luminance, In-phase, Quadrature)

- Used in analog televission broadcasting (NTSC TV system in the USA and JAPAN)
- Separates brightness (Y) from color info, enabling backward compatibility with black and white TVs.

HSI (Hue, Saturation, intensity)

- Used in image analysis, computer vission and graphics
- Helpfull in Object recognization, color based segmentation, and enhancement since it matches human color perception.

e) Approches for handling image borders in filtering.

1. Zero padding (Constant Extension)
   - Assume missing neighbours are '0' (black)
   - Simple, but can introduce dark artifacts at edge.

2. Replicate padding (Edge replication).

* Extend the border pixels outward (Copy nearest edge value)
* preserve edge brightness better

3. Mirror/Reflect padding

* Reflect the image values at the border
* Reduce edge discontinuity, commonly used in OpenCV

4. Wrap Around

* Treat image as if it wraps around - left border connects to right, top to bottom.
* Works well for periodic signals, less natural for image.

(5) Group and Output :-

* perform filtering only where the kernel fully overlaps the image.
* Avoids padding but reduces the image size.

6. Extend with Mean/Constant Value

(*) Extend boarders with average pixel intensity or a chosen constant

(*) Can minimize sudden jumps at edge

(f) Lookup Table :- A LUT in image processing is a predifined array that maps input pixel values to output values.

Instead of computing a transformation for each pixel, we store the result in a table and directly replace pixel values using the LUT

FFT (Fast Fourier Transform) : The FFT is an effient algorithm to compute the discrete fourier transform.

* DFT transform an image from the spacial domain to the frequency domain.
* FFT reduces the complexity from $O(N^2)$ to $O(N \log N)$, making it practical for large images.

2-D Moment (of an image)

A moment is a weighted average of pixel intensity, used to describe the shape and distribution of an image.

* 2-D Moment of order (p,q) is defined as

$$M_{pq} = \sum_x \sum_y x^p y^q f(x,y)$$

Types of moment

1. Raw moment
2. Central moment
3. Moment invariants —

Ans: to the Question no: 2

a) Unsharp Masking vs High bost filtering

Aspect	Unsharp Masking	High bost filtering
purpose	To enhance edge and fine details by substructing a blurred version of an image	To enhance both edge and retain the original image intensity
Technique	Substruct a blurred version (low pass) from the original image	Adds a scale version of the image to the unsharp mask
Formula	Sharpened = original − blurred	High bost = A * original − Blurred, where A > 1
Weight of original	The original image is not weighted	The Original image is multiply by a constant A (>1)

Aspect	Unsharp Masking	Highboost filtering
Where A=1	Equivalent to Unsharp masking	High-boost filtering becomes Unsharp masking
Use case	Enhancing edge constrast in moderately detailed image	Sharping while preserving more of the image content.
Edge Enhancement	Medium enhancement	Stronger and adjustable enhancement using A
Output Image	Might look less natural if overdone	provides more control over sharpness and brightness

ⓑ Source of the noise of digital image

1. Image Aquisition (Sensor noise)
2. Transmission noise
3. Environmental Condition

4: Digitization process
5. processing Artifacts

Noise model in digital image

Noise is usually modeled statistically because it is random. The most common models:

1. Gaussian Noise

probability distribution:

$$P(z) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(z-\mu)^2}{2\sigma^2}}$$

Here, μ is mean (often 0), σ^2 is varience

Source: Sensor thermal noise, electric circuit noise.

Effect: adds smooth random variation in intensity.

2. Salt and papper Noise (impulse noise)

* Random occurance of black and white pixels

probability distribution:

$$P(z) = \begin{cases} P_a & z = a \text{ (black)} \\ P_b & z = b \text{ (white)} \\ 1-(P_a+P_b) & \text{otherwise} \end{cases}$$

Source: Faulty pixels in sensors, bit error in transmission

Effect: appears as scattered white/black

3. poisson Noise (Shot noise)

. Caused by the discreate nature of light

probability distribution:

$$P(z) = \frac{\lambda^z e^{-\lambda}}{z!}$$

Source: Low light condition in image aquisition

Effect: Intensity dependent noise-

4. Speckle Noise

Multiplicative noise, modeled as:

$$g(x,y) = f(x,y) + f(x,y) \cdot n(x,y)$$

where $f(x,y)$ is the true image, $n(x,y)$ is noise

Source: Coherent imaging system like radars, ultrasound, SAR

Effect: grainy appearence in image

General Noise model

An observe noisy image $g(x,y)$ is modeled as:

$$g(x,y) = f(x,y) + \eta(x,y)$$

Where:

$f(x,y)$ = original image

$\eta(x,y)$ = additive noise

(c) Basic principle of:

(1) Histogram Equalization (HE)

principle: Redistributed the pixel intensity values so that the histogram of the output image is Uniform

* This spreads out the most frequent intensity values → increase global contrast

* Steps:

1. Compute histogram
2. Calculate CDF
3. Map input pixel intensities to new intensities using CDF

(2) Histogram modeling:

principle: Modify the image histogram to fit the desire shape.

(*) Unlike equalization, the goal is here is not Uniform distribution but shopping it for specific enhancement

3. Histogram Specification

principle:- Transform the image such that it's histogram matches a specified target histogram.

(*) Useful when you want the output image to look similar to a reference image in term of brightness/Constrast.

Steps:

1. perform HE on input Image
2. perform HE on tagged image
3. Use mapping between two equalization histogram to remap pixel intensity-

(d) Relationship between RGB, HSI, and CMYK:

(1) RGB → HSV

Let (R, G, B) be the normalized red, green, blue components in the range ([0, 1]). So if they are are between 0 and 255, we first divide each value by 255. We then defined

$$V = \max(R, G, B)$$
$$\delta = V - \min(R, G, B)$$
$$S = \frac{\delta}{V}$$

To obtain value of hue, we consider several cases:

1. if $R = V$ then $H = \frac{1}{6}\frac{G-B}{\delta}$
2. if $G = V$ then $H = \frac{1}{6}\left(2 + \frac{B-R}{\delta}\right)$
3. if $B = V$ then $H = \frac{1}{6}\left(4 + \frac{R-G}{\delta}\right)$

2. HSV → RGB

$$H' = \lfloor 6H \rfloor$$
$$F = 6H - H'$$
$$P = V(1-S)$$
$$Q = V(1-SF)$$
$$T = V(1-S(1-F))$$

Since H' is an integer between 0 and 5 we have six cases to consider

H'	R	G	B
0	V	T	P
1	Q	V	P
2	P	V	T
3	P	Q	V
4	T	P	V
5	V	P	Q

3. RGB → CMYK

$$C = 1 - R$$
$$M = 1 - G$$
$$Y = 1 - B$$

Extract K (Black)

$$K = \min(C, M, Y)$$

Adjust CMY values

$$C' = \frac{C-K}{1-K}$$

$$M' = \frac{M-K}{1-K}$$

$$Y' = \frac{Y-K}{1-K}$$

So, final $CMYK = (C', M', Y', K)$.

4. CMYK $\rightarrow$ RGB

$$R = (1-C) * (1-K)$$

$$G = (1-M) * (1-K)$$

$$B = (1-Y) * (1-K)$$

Ans: to the Question no: 3

a) Otsu method ~~and~~ or adaptive thresholding

Otsu's Method:– Otsu's methods is global thresholding technique used to automatically find the best thresholding value to separate an image into foreground and background

Instead of selecting threshold manually, Otsu's algorithom finds it by minizing the intra-class varience –

How it works:

1. Histogram based approch:

Compute the histogram of the gray scale image.

* Each bin corresponds to the probability of that intensity

2. <u>Threshold Canditate</u>

* Try every possible threshold t (0 to 255)
* At each t, split pixel into two classes.

C_1 : pixel with intesity $[0, t]$ (background)

C_2 : pixel with intensity $[t+1, 255]$ (forground)

3. <u>Compute class means</u>

$\mu_1(t)$ = mean intensity of class C_1

$\mu_2(t)$ = mean intensity of class C_2

4. <u>Compute class probabilities</u>

$\omega_1(t)$ = probability of class C_1

$\omega_2(t) = 1 - \omega_1(t)$. probability of class C_2

5. Between class varience

Otsu's defines the best threshold as the one that maximizes:

$$\sigma_b^2(t) = \omega_1(t)\,\omega_2(t)\left(\mu_1(t)-\mu_2(t)\right)^2$$

6. Optimal threshold

choose the threshold t^* that maximuze of $\sigma_b^2(t)$.

(b) (i) Laplacian filtering technique

The laplacian filter is a second derivative filter used in digital image processing to highlight region of rappid intensity change.

It is based on the laplacian operator

from mathematics

$$\nabla^2 f(x,y) = \frac{\delta^2 f}{\delta x^2} + \frac{\delta^2 f}{\delta y^2}$$

Kernel Examples

Discreate approximations of the laplacian operator are useally implemented with small convolution kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix} \quad \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

- positive values highlist edges
- Zero-sum kernel ensure uniform areas vanish

Application

1. Edge detection
2. Sharpening images
3. Feature Extraction

(ii) Derivative Filter

A derivative filter computes the rate of change of pixel intensity in an images it based on the concept of first derivative in calculus.

In 2D images:

- **First derivative**: Detect edge by highlighting transition in intensity
- **Second derivative**: Detects changes in edges.

Example of derivative filter

1. **First order derivative filters**

- Sobel filter
- prewitt filter
- Roberts cross filter

→ these approximate $\frac{\delta f}{\delta x}$ and $\frac{\delta f}{\delta y}$

Example Sobel Kernel (for x-gradient):

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

2. Second-order derivative filter

- Laplacian filter

Applications

- Edge detection
- Texture analysis
- Feature etraction

(c) Homo-morphic transformation:-

Concept:

Digital images are often modeled as product of two components:

$$f(x,y) = i(x,y) \cdot r(x,y)$$

* $i(x,y)$ = Illumination
* $r(x,y)$ = Reflection/tance

problem:

* poor illumination reduce visibility
* Simply stretching contrast may enhance illumination and noise together

Solution: Homomorphic filtering

* Separate illumination and reflectance
* Support low-frequency illumination
* Enhance high frequency reflectance

Step homomorphic filtering:-

1. Log Transformation

Since $f(x,y) = i(x,y) \cdot r(x,y)$

Apply log:

$$g(x,y) = \ln(f(x,y)) = \ln(i(x,y)) + \ln(r(x,y))$$

2. Fourier Transform

$$G(u,v) = I(u,v) + R(u,v)$$

3. Filtering

Apply a filter $H(u,v)$ that:

- Attenuates low-frequency component
- Amplifies high-frequency components

A high pass filter is often used.

4. Inverse Fourier Transform

Convert ~~Fourier~~ filtered spectrum back to spacial domain

5. Exponential Transformation:

Undo log transform

$$f(x,y) = e^{g(x,y)}$$

Result = Enhanced image with balanced illumination + Sharper details.

Application:

1. Correct uneven illumination in images
2. Medical image enhancement
3. Face recognization
4. Satellite image.

(d) Image Segmentation Technique:-

Image segmentation technique is the process of dividing an image into meaningful regions or objects. It helps identity shapes, boundary, or regions of interest for further analysis.

Main technique of image segmentation

1. Thresholding-based Segmentation:-

* Simplest method
* Separate objects from background using pixel intensity
* Example: Otsu's methods
* Works well for images with distinct intensity difference between objects and background

ent

2. Edge-based Segmentation

* Detects objects boundaries using gradient operators like, Sobel, prewitt, canny.
* Relies on changes in intensity to separate objects
* Suitable for objects with well-defined boundaries

3. Region based Segmentation

* Group pixels into regions based on Similarity (intensity, texture, color)

* Useful when objects have homogeneous properties

4. Clustering-based Segmentation

* Treats segmentation as a Clustering problem

* Example: k-means clustering

* Commonly used for color image segment

Application:

1. Medical imagin
2. Object detection
3. Remote Sensing
4. Industrial inspection

Ans: to the Question no: 4

a) Image Compression Method

Image compression reduce the size of image data of storage or transmission - It is classified into two main types:

1. Lossless Compression

- No info is lost
- The original image can be exactly reconstruct
- used where accuracy is critical
- Example: RLE, TIFF, PNG

2. Lossy Compression

- Some info is lost
- Achieve very high compression ratios
- Used in multimedia
- Example: JPEG, MPEG

Run-Length Encoding

principle:

RLE compresses data by replacing conse-cative repeating values with a single value with count.

it is efficient when images contain large Uniform areas.

Example:

Suppose we have 1D image row:

pixels: 00000 111 00 2222

RLE: (0,5), (1,3) (0,2) (2,4)

Advantages:

- Very simple and efficient for image with large uniform areas.

Disadvantages: Inefficient for highly detailed or noisy image.

b)

1) proof that the first-order central moment $= 0$

Definition:

Let $f(x,y)$ be the image intensity and sum over all pixel coordinations

⊛ Raw (non-central) moment:

$$M_{pq} = \sum_x \sum_y x^p y^q f(x,y)$$

⊛ Zeroth moment (total mass/area/sum of intensity)

$$M_{00} = \sum_x \sum_y f(x,y)$$

⊛ Image centroid (mean coordinates):

$$\bar{x} = \frac{M_{10}}{M_{00}}, \quad \bar{y} = \frac{M_{01}}{M_{00}}$$

⊛ Central moments:

$$\mu_{pq} = \sum_x \sum_y (x-\bar{x})^p (y-\bar{y})^q f(x,y)$$

We want to show $\mu_{10} = 0$ and $\mu_{01} = 0$

proof for μ_{10}

By definition:

$$\mu_{10} = \sum_x \sum_y (x-\bar{x}) f(x,y) = \sum_x \sum_y x f(x,y) - \bar{x} \sum_x \sum_y f(x,y)$$

The first double summ is M_{10} and the second is M_{00}, so

$$\mu_{10} = M_{10} - \bar{x} M_{00}$$

But $\bar{x} = M_{10}/M_{00}$, hence

$$\mu_{10} = M_{10} - \frac{M_{10}}{M_{00}} M_{00} = M_{10} - M_{10} = 0$$

Same steps for μ_{01}:

$$\mu_{01} = \sum_x \sum_y (y-\bar{y}) f(x,y) = M_{01} - \bar{y} M_{00} = M_{01} - \frac{M_{01}}{M_{00}} M_{00}$$
$$= 0$$

So, $\mu_{10} = \mu_{01} = 0$

2) Why we generally donot use arbitrarily higher or very low order moments in image processing?

Low order moment

* Zeroth moment M_{00} is useful: gives total area.
* First-order raw moment (M_{10}, M_{01}) give centroid but centralizing removes them. So first-order central moments carry no additional shape information – hence they are not

used as discriminative descriptors.

Very high-order moments – problems and reason to avoid :-

1. Noise sensitive : higher-order moments weighted coordinates with high powers $(x-\bar{x})^p$ Small pixel noise for from centroid is amplified heavily → Unstable feature –

2. Newmerical instability :– Large powers lead to very Large/Small numbers, causing floating point errors and loss of precision

3. poor robustness to geometric perturbation

4. Diminishing return

5. Computational Cost

For reason high and low order moment don't use –

c) Thinning Algorithm

Definition: Thinning is a morphological operations that reduces binary image objects to their skeletons, without breaking their connectivity. It is often used in OCR, fingerprint recognization, shape analysis.

Algorithm (Zhang-Suen Thinning Algorithm)

This is one of the most common iterative thinning algorithm.

Input: A binary image (1=Forground, 0=background)

Steps:

1: Repeat untill no more flexible pixels can be removed:

Step-1

For each pixel P_1 in the image, with neighbours $P_2 --- P_9$ arranged as

$$\begin{bmatrix} P_9 & P_2 & P_3 \\ P_8 & P_1 & P_4 \\ P_7 & P_6 & P_5 \end{bmatrix}$$

⊛ Condition for deleting P_1:

1. $2 \leq N(P_1) \leq 6$

Where $N(P_1)$ = number for non-zero neighbours among P_2 --- P_9

2. $S(P_1) = 1$; Where $S(P_1)$ = number of $0 \to 1$ transitions in the ordered Sequence $[P_2\, P_3 \cdots P_9 \cdots P_2]$

3. $P_2 \cdot P_4 \cdot P_8 = 0$

4. $P_2 \cdot P_6, P_8 = 0$

If all condition is satisfied, mark P_1 for deletation.

Step-2

· Again for each pixel P_1, check:

1. $2 \leq N(P_1) \leq 6$

2. $S(P_1) = 1$

3. $P_2 \cdot P_4 \cdot P_8 = 0$

4. $P_2 \cdot P_6, P_8 = 0$

if satisfied mark P_1 for deletion

* Remove all pixels marked in both steps
* Repeat steps 1 and 2 until no further pixels can be deleted

Output: A thinned binary image.

d) (i) **Prewitt Mask**

* The prewitt operator is used for edge detection.
* It uses two 3x3 convolution masks to detect horizontal and vertical edges.

Horizontal edges

$$\begin{bmatrix} -1 & 0 & +1 \\ -1 & 0 & +1 \\ -1 & 0 & +1 \end{bmatrix}$$

vertical edges

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ +1 & +1 & +1 \end{bmatrix}$$

ii) Laplacian Mask

- The laplacian operator is a second derivative filter used for detecting edge and regions of rapid intensity change.

- Commonly used Laplacian Mask (3X3):

4. connected version

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

8. connected version

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Prewitt $\rightarrow$ 2 mask (horizontal, vertical)

Laplacian $\rightarrow$ 1 Mask (second derivative)

Ans: to the Question no: 5

(5-a-i)

Cumulative Distributive Function → Cumulative Count and normalized

Value	Count	Cumulative	normalized-eDF
0	10	10	10/42 = 0.238
2	9	19	19/42 = 0.452
3	2	21	21/42 = 0.500
4	10	31	31/42 = 0.738
5	3	34	34/42 = 0.810
6	1	35	35/42 = 0.833
10	3	38	38/42 = 0.905
20	2	40	40/42 = 0.952
30	1	41	41/42 = 0.976
31	1	42	42/42 = 1

Comment on quality

* Converting 5-bit → 8-bit by Linear scale multiplies each value by $255/31 = 8.2258$. this spreads the limited existing gray levels into a wider 0-255 range but does not add new information

(5-a-ii) Binary conversion with threshold = 15

Threshold rule: pixel → 1 values ≥ 15 else 0

Applying to the matrix yeilds the binary 6x7 result:

Binary matrix (Threshold 15):

```
[[0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0, 0],
 [0, 1, 0, 0, 0, 0, 0], # 20→1
 [1, 0, 0, 0, 0, 0, 0], # 30→1
 [0, 0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 1, 1, 0]] # 20, 31 → 1
```

(5-b) Steps of PCA

① **Collected Data**:- prepare a dataset in matrix form.

2. **Standardize/Normalize**:- Substruct the mean of each feature and scale if the units differ.

3. **Compute Covariance Matrix**:
Calculate the covariance matrix of the Standardized dataset.

$$C = \frac{1}{N-1} X^T X$$

4. **Find Eigenvalues and Eigenvector**:- perform eigen~~values~~ ~~a~~decomposition of the covarience matrix to find eigen-values and eigenvectors.

5. **Sort eigen values**:- Rank eigenvectors according to descending eigenvalues-

6. Select to k components!

Choose the top k eigenvectors to reduce dimensionality

7. Transform data:

project the original data onto the new Subspace formed by these k eigenvectors:

$$Y = XW$$

Where W is the matrix of selected eigenvectors

8. Evaluate performance

Check the percentage of total variance retained:

$$\text{Explained Variance Ratio} = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{j=1}^{d} \lambda_j}$$

(5-c) Fig-2 binary image

Original (5x5):

R1: 0 0 0 1 1
R2: 0 1 1 0 1
R3: 0 1 0 0 1
R4: 1 1 1 1 1
R5: 1 1 0 1 1

Structuring element (3x3 cross):

SE (cross) = [
[0, 1, 0],
[1, 1, 1],
[0, 1, 0]]

Erosion rule: a pixel remains 1 in the eroded image only if all SE positions that are 1 are also 1 in the corresponding neighbourhood.

Result of erosion:

Eroded image:-

[[0, 0, 0, 0, 0],
[0, 0, 0, 0, 0],
[0, 0, 0, 0, 0],
[0, 1, 0, 0, 0],
[0, 0, 0, 0, 0]]

So after erosion with the cross SE, a single pixel remains 1 at row 4 column 2. All other pixels become 0.